Design, Streaming and Scale

Topic 7 of 10

18 questions | skeleton and writing space only

Each question: the prompt, the function or class signature, and ruled lines to write your own solution.

1. Implement LRU Cache.

HARD | asked n/a |

Your program is running slowly because it's accessing data from disk over and over again. To improve the performance, you want to build a simple key-value store to cache this data in memory, but you also want to limit the amount of memory used. You decide to build a caching system that only keeps the N most recently used items--also known as a least recently used (LRU) cache. Write a class LRUCache(n) that accepts a size limit n.

```
class LRUCache:

    def __init__(self, capacity):

    def unlink(self, node):

    def push_front(self, node):

    def get(self, key):

    def put(self, key, value):
```

2. Implement a function that processes a stream of interval data.

HARD | asked n/a |

Implement a function that processes a stream of interval data.

```
class IntervalSet:

    def __init__(self, integer=True):

    def length(self, start, end):

    def add(self, start, end):

    def contains(self, point):

    def total(self):

    def intervals(self):
```

3. Implement a Reusable Locker Allocation System

MEDIUM | asked 2026-08-14 |

Problem A locker bank has lockers numbered from 1 through n. Process operations in order. store(customer) assigns the smallest available locker and returns its number, or -1 if that customer already has a locker or none is available. pickup(customer) releases and returns that customer's locker, or -1 if no locker is assigned. last() returns the locker number from the most recent successful store, or -1 if there has not been one. Return one result per operation.

```
class Lockers:

    def __init__(self, n):
```

```
    def store(self, customer):
```

```
    def pickup(self, customer):
```

```
    def last(self):
```

4. Build Prefix Lookup with a Trie

MEDIUM | asked 2026-07-03 |

Overview: Practice a Google coding interview problem focused on build prefix lookup with a trie.

```
class PrefixTrie:

    def __init__(self, words=()):
```

```
    def insert(self, word):
```

```
    def undo_count(self, word):
```

```
    def walk(self, prefix):
```

```
    def count_prefix(self, prefix):
```

```
    def root_count(self):
```

```
    def suggest(self, prefix, k=3):
```

```
    def completions(self, prefix):
```

5. Design a restaurant waitlist system

MEDIUM | asked 2026-03-01 |

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available. ## Rules - Each party has a unique partyId, a name, and a size (number of people). - Parties are seated in **arrival order, but only if they **fit** the available table. - When a table of capacity c becomes available, you must seat the **earliest-arrived** party whose size <= c.**

```
class Waitlist:

    def __init__(self, capacity=1024):
```

```
    def update(self, i, value):
```

```
    def add(self, party_id, name, size):
```

```
    def cancel(self, party_id):
```

```
    def seat(self, capacity):
```

6. Determine Reporting Relationships

MEDIUM | asked 2026-02-22 |

Build an in-memory organization model that supports three operations on employee IDs: 1. add_manager(a, b): employee a is the direct manager of employee b. 2. add_peer(a, b): employees a and b share the same direct manager. 3. is_manager(a, b): return whether employee a is a manager of employee b. The answer should account for relationships that can be inferred from prior operations.

```
class Org:

    def __init__(self):
```

```
    def find(self, x):
```

```
    def add_peer(self, a, b):
```

```
    def add_manager(self, a, b):
```

```
    def manager_of(self, x):
```

```
    def is_manager(self, a, b):
```

```
    def common_manager(self, a, b):
```

7. Implement Trie Insert and Exact-Word Search

MEDIUM | asked 2026-02-20 |

Problem Implement a trie that supports insert(word) and search(word). Insert adds a lowercase word. Search returns true only when the complete word was previously inserted; a stored word merely sharing the requested prefix is not a match. Process a sequence of operations and return the results of all searches. ### Constraints & Assumptions - There are at most 200,000 operations. - Every word has length between 1 and 1,000.

```
class Trie:

    def __init__(self):

    def walk(self, text):

    def insert(self, word):

    def search(self, word):

    def starts_with(self, prefix):

    def search_wild(self, pattern):

    def delete(self, word):
```

8. Design Tic-Tac-Toe With K Players

MEDIUM | asked 2026-02-12 |

Design and implement a Tic-Tac-Toe game engine. You are given: - k players, each with a unique player ID or mark. - An n x n board, initially empty. - Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large n is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left.

```
class TicTacToe:

    def __init__(self, n, k, target=3):

    def turn(self):

    def run_length(self, r, c, dr, dc, player):

    def move(self, player, r, c):

    def undo(self):
```

9. Restaurant Waitlist: join / delete / find_first_match

MEDIUM | asked 2026-02-01 | data-structure, oop-design

Onsite coding round: design and implement a restaurant waitlist data structure supporting join, delete, and find_first_match(table_size). The find call returns the earliest-joined user whose party size fits the given table. Implement a class with these methods: join(user) -- append user (with their party_size) to the waitlist. delete(user) -- remove user from the waitlist (could be anywhere in the queue).

```
class Waitlist:

    def __init__(self):
```

```
    def join(self, name, size):
```

```
    def delete(self, name):
```

```
    def front(self, queue):
```

```
    def find_first_match(self, table):
```

10. Find top/bottom-k words in list or stream

MEDIUM | asked 2026-01-11 |

You are given words (strings) either as a finite list or as an unbounded stream. 1) **List version**: Given an array words and an integer k, return: - the k most frequent distinct words, and - the k least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order).

```
def top_bottom(words, k):
```

11. Design a logger that returns top-K messages

MEDIUM | asked 2026-01-06 |

Overview: This question evaluates skills in designing in-memory data structures and algorithms for frequency counting and interval querying, including extracting top-K elements with a heap, within the Coding & Algorithms domain.

```
class Logger:

    def __init__(self):
```

```
    def log(self, ts, msg):
```

```
    def top_k(self, k, start=None, end=None):
```

```
    def expire(self, before):
```

12. Implement iterator merging lists with filtering

MEDIUM | asked 2026-01-06 |

Overview: This question evaluates iterator/generator design, sequence merging, stateful filtering and deduplication, checking a candidate's ability to maintain input ordering while excluding blocked IDs and preventing duplicate yields.

```
class FilteredZigzag:

    def __init__(self, lists, blocked=()):
```

```
    def advance(self):
```

```
    def has_next(self):
```

```
    def next(self):
```

13. Design set with O(1) random access

MEDIUM | asked 2026-01-01 |

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time: - add(x): Insert value x if it is not already present. Return whether the insert happened. - delete(x): Remove value x if it exists. Return whether a deletion happened. - getRandom(): Return a uniformly random element currently in the set. - Handle corner cases such as calling getRandom() on an empty set (define and justify behavior).

```
class FancySet:

    def __init__(self, rng=None):

    def add(self, x):

    def delete(self, x):

    def get_random(self):
```

14. Match people to questions using tags and priorities

MEDIUM | asked 2025-12-27 |

Overview: This question evaluates competencies in bipartite matching and combinatorial optimization, along with practical data-structure design for scalable tag-based assignment and priority-aware selection.

```
def max_matching(adj):
```

15. Design server allocation for multi-type nodes

MEDIUM | asked 2025-09-06 |

Design a service to manage a large pool (about 100, 000) of server nodes. Each node can satisfy multiple instance types; clients call getServer(reqNumOfServer, instanceTypes[]) to request servers that collectively meet the requested types, and returnServer(ids[]) to release them. Propose data structures and indexes for fast matching, availability tracking, fragmentation minimization, and scaling. Address concurrency, starvation, and failure handling.

```
class ServerPool:

    def __init__(self, servers):


    def best_for(self, t, skip):


    def get_server(self, count, types):


    def release(self, ids):


    def rent(self, count, types, now, ttl):


    def expire(self, now):
```

16. Design an elevator control system

MEDIUM | asked 2025-09-06 |

Design the software for a multi-elevator control system in a high-rise building. Specify classes, scheduling algorithms for car assignments, handling peak traffic, fairness, fault tolerance, and safety constraints. Consider real-time constraints and a simulation and testing strategy.

```
class Elevator:

    def __init__(self, car_id, floor=0):


    def cost(self, req):


    def step(self):
```

17. Hit Counter

MEDIUM | asked n/a |

Design a hit counter that counts the number of hits received in the past 5 minutes (300 seconds). hit(timestamp) records a hit; getHits(timestamp) returns the number of hits in the past 300 seconds.

```
class QueueCounter:

    def __init__(self):
```

```
    def hit(self, ts):
```

```
    def get_hits(self, ts):
```

18. Implement Batched Undo/Redo Layer

EASY | asked 2026-02-04 |

You are implementing a simplified document layer for a design tool. The layer stores properties as a map<string, string>. Implement a class that supports the following operations: - beginBatch(): start a batch of edits. - setProperty(key, value): set or overwrite a property inside the current batch. - endBatch(): commit the batch as one undoable action. - undo(): revert the most recently committed batch. - redo(): reapply the most recently undone batch. Requirements: 1.

```
class Layer:

    def __init__(self, history=None):
```

```
    def begin_batch(self):
```

```
    def set_property(self, key, value):
```

```
    def end_batch(self):
```

```
    def merge(parent, child):
```

```
    def undo(self):
```

```
    def redo(self):
```